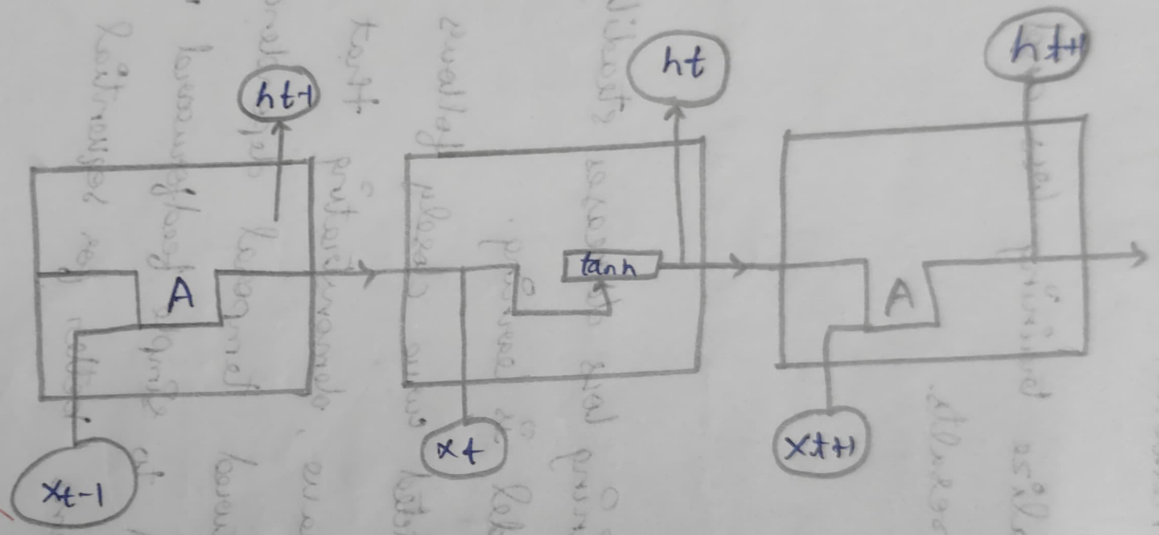
RNN Architecture :-



Exp: 9. BUILD A RECURRENT NEURAL Network

AIM:

To Build a Recurrent Neural Network.

OBJECTIVE:

* To understand and implement the working of an RNN layer in Pytorch.

* To train the network on sequential data and predict future values based on past inputs.

* To observe how RNNs capture temporal dependencies in the input sequences.

Epochs:

Epochs 1/5 Train loss : 0.6432 Test loss : 0.6...

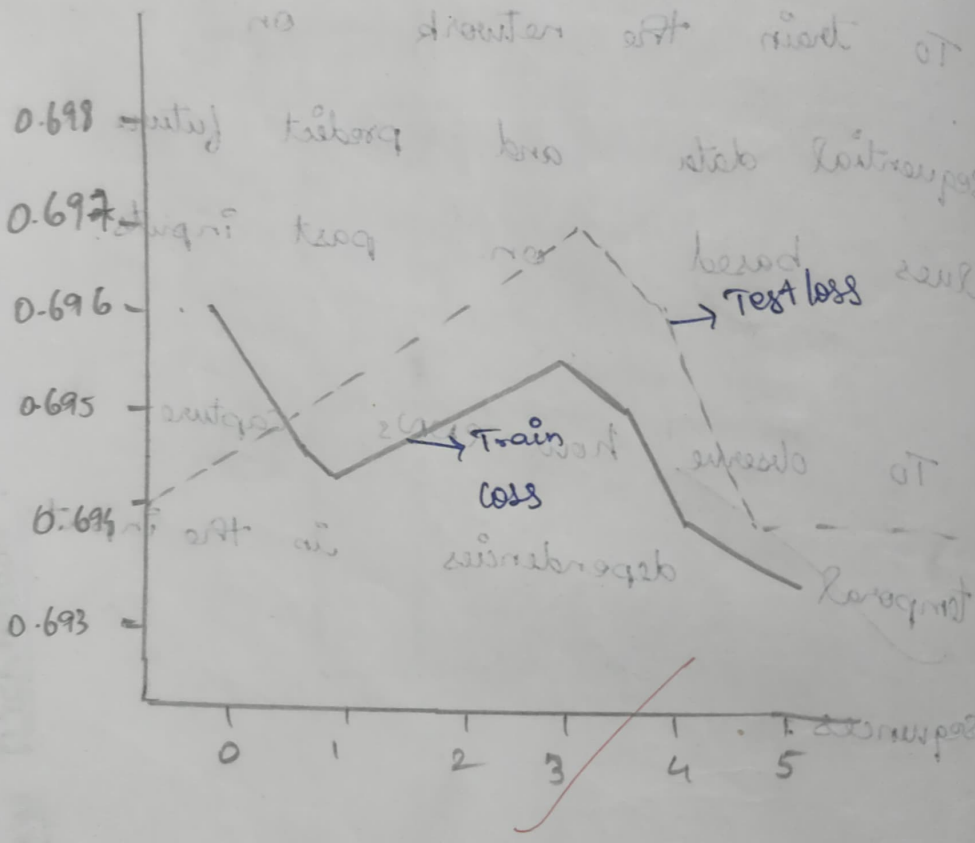
Epoch 2/5 Train loss : 0.6435 Test loss : 0.6...

Epoch 3/5 Train loss : 0.6021 Test loss : 0.6...

Epoch 4/5 Train loss : 0.6632 Test loss : 0.6...

Epoch 5/5 Train loss : 0.6015 Test loss : 0.6...

Test / Train loss Graph :



PSEUDOCODE :-

1. Import torch, nn, optim.

2. Prepare dataset (x, y) as tensors.

3. Define RNN model :

```
class RNN (nn.Module):
  def __init__():
  self.rnn = nn.RNN (input_size, hidden_size);
  self.fc = nn.Linear (hidden_size, output_size).
  def forward (x):
  h0 = zeroes (1, batch, hidden size)
out, _ = self.rnn (x.h0)
return self.fc (out [:, -1, :])
```

4. Initialize model, loss (MSE), optimizer. (Adam)

5. For each people epoch :

```
output = model (x)
loss = criterion (output, y)
optimizer.zero_grad ().
loss.backward
Optimiser.step()
```

6. Evaluate model on test data

OBSERVATION:-

* The RNN model effectively captures short-term dependencies in sequential data.

* Training stability and convergence depend on proper initialization, sequence length and learning rate.

* For longer sequences vanishing gradient may occur, suggesting the use of LSTM / GRU as better alternatives.

RESULT:-

Therefore a Recurrent Neural Network has build successfully.